

FC-MAGICON 仕様書(ハードウェア・ソフトウェア総合)

FC-MAGICON のハードウェア・ファームウェア・PC のソフトウェア・通信とデータの形式・試験の記録を 1 冊にまとめたもの。2026-09-27 時点。
基板 rev0.2 は発注済み・未着で、ファミコンの実機では未検証。ファームウェアと PC のソフトは、PC の試験台(sim)と Core2350B2 単体
(ピンヘッダーを付けたモジュール)で確かめた範囲まで。

目次

1. 概要	2
1.1 全体の構成	2
1.2 版と状態(2026-09-27)	3
2. 動作モード	3
3. ハードウェア	3
3.1 構成	3
3.2 主要部品	4
3.3 電源	4
3.4 バスの接続と電圧	4
3.5 PPU アドレスの読み取り方	5
3.6 制御・周辺回路	5
3.7 デバッグ・テスト用	6
3.8 機械仕様	6
3.9 基板の製造仕様	6
3.10 ファームウェアが守ること	7
3.11 モジュール単体の試験で使ったピン(2026-09-27)	7
4. ファームウェア(RP2350)	7
4.1 ファームウェアの一覧	7
4.2 作りの全体	8
4.3 起動の流れ	8
4.4 CPU バスの応答	8
4.5 PPU バスの応答	9
4.6 マッパー	9
4.7 NSF プレイヤー	9
4.8 画面転送(リモートデスクトップ)モード	10
4.9 PC の音	10
4.10 ゲーム選択メニュー	11
4.11 本体の電源の見張り	11
4.12 USB	11
4.13 LED と表示	12
5. Wi-Fi ブリッジ(ESP32-C6)	12
6. PC のソフトウェア	12
6.1 道具の一覧	12
6.2 remote_pc.py	13
6.3 画面の変換(nesframe.py)	13
6.4 PC の音(pcaudio.py)	14
6.5 ROM の書き込み	14

6.6 キーボードの割り当て	14
7. データと通信の形式	15
7.1 フラッシュの使い方	15
7.2 ROM 置き場: 1 本(FCMG)	15
7.3 ROM 置き場: ライブラリ(FCLB)	15
7.4 画面転送の通信	15
7.5 画面のデータの並び	16
8. 試験と検証の道具	16
9. 性能(測った値)	17
10. 開発環境	17
11. 制限と未実装	18
12. 確認状況	18
13. 部品表	19
付録 A. ピン・接続の一覧	20
カセット端子 J1(60ピン)	20
Core2350B のピン(ヘッダー P1～P4)	21
J4 ブレイクアウト(2x30)	23
J5 デバッグ(2x8)	24
周辺部品の接続	24
付録 B. ソフトウェアの規模	26
関係するファイル	26

1. 概要

ファミコン(HVC-001、本体基板 HVC-CPU-07)のカセットスロットに挿す自作カセット。Waveshare Core2350B2(RP2350B)1 個が、CPU のバスと PPU のバスの両方に直接応答する。FPGA は使わない。

機能	内容
マジコン	フラッシュに書いた .nes を SRAM に写し、PRG(CPU 側)と CHR(PPU 側)を返す。マップ 8 種類(4.6)
ゲーム選択メニュー	何本もまとめて書き、起動時にファミコンの画面で選ぶ(4.10)
NSF プレイヤー	.nsf を本体の 2A03 で鳴らす。曲の切り替え・一時停止(4.7)
画面転送(リモートデスクトップ)	PC の画面を 256 x 240 ドットでファミコンに映す。1コン = マウス、ファミリーベーシックのキーボード = PC のキーボード(4.8)
PC の音	画面転送中、PC で鳴っている音をファミコンの音声に混ぜて鳴らす(4.9)
Wi-Fi	ESP32-C6 を J6 につなぎ、画面転送を Wi-Fi で行う(5 章)

1.1 全体の構成

所	部品・ソフト	役目
PC(Windows 11)	remote_pc.py・pcaudio.py・nesframe.py・load_rom.ps1 など(6 章)	画面の取り込みと変換、音の横取り、入力をマウス・キーへ、ROM の書き込み
通り道 1	USB(Core2350B2 の FPC-USB アダプター)	PC 直結。614KB/秒
通り道 2	Wi-Fi → ESP32-C6(TCP ⇄ USB の素通し) → J6 → RP2350(USB ホスト)	無線。420KB/秒
カセット	RP2350B(コア 0 = USB・画面転送・音、コア 1 = バスの応答)、PIO 3 つ	バスに応答、ROM・画面・音を出す
ファミコン	2A03(CPU・音源)、2C02(PPU)、本体 RAM・VRAM	6502 のプログラム(ゲーム、または FC-MAGICON が用意するドライバ)を動かす

1.2 版と状態(2026-09-27)

もの	版	状態
基板	rev0.2(kicad/FC-MAGICON_gerber_r0.2.zip)	JLCPCB に発注済み・未着。2 層、黒、1.2mm
マイコン	Waveshare Core2350B2(RP2350B A4、QFN-80)	手元にあり。ピンヘッダーを付けて単体で試験中
ファームウェア	pico-sdk 2.3.1	ビルド済み。PC の試験台と単体で確認した範囲まで(12 章)
Wi-Fi ブリッジ	ESP32-C6 N4、Arduino-ESP32 3.3.12	動作確認済み(PC 相手・RP2350 相手)
ケース	printables 860420 のシエル(CC BY 4.0)を改造	ユーザーが編集中。3D プリンターは Bambu Lab P1S
シール	case/label1/(99 x 60.6mm)	L 判の光沢紙に印刷済み

2. 動作モード

起動した時にフラッシュの ROM 置き場の中身で決まる。

モード	入り方	カセットがすること	本体の 6502 が動かすもの	USB(magicon)
ゲーム	load_rom.ps1 game.nes、またはメニューで選ぶ	PRG・CHR・WRAM を返し、マップを真似る	ゲーム	1 秒ごとの様子を USB シリアルに
NSF プレイヤー	load_rom.ps1 music.nsf、またはメニューで選ぶ	曲のデータとドライバーを返し、ベクターを横取り	NSF ドライバー (\$5000-)と曲	同上
画面転送	load_rom.ps1 -Remote、またはメニューの REMOTE DESKTOP	PPU の読み出しに PC の絵を返す。音を鳴らす	画面転送ドライバー (\$C000-)	画面・音を受け、入力を返す
ゲーム選択メニュー	load_rom.ps1 -Library フォルダー	メニュー(NROM)をその場で組み立てて返す	メニュー(\$C000-)	同上
試験画面	ROM が無い・壊れている・対応していない時	chr_test と同じ 4 色の縦縞 + 格子と音	試験 ROM	理由を USB シリアルに

別のファームウェア(bus_test・chr_test)は、基板が届いて最初に確かめる時だけ使う(4.1)。

3. ハードウェア

基板の詳しい所は docs/hardware_spec_body.md をそのまま取り込んでいる(3.1～3.10)。3.11 は、モジュール単体の試験で使ったピン。

3.1 構成

経路	内容
CPU バス(26 本)	カセット端子 → Core2350B の GPIO0～25 に直結(D0～D7、A0～A14、/ROMSEL、R/W、M2)
PPU バス	AD0～AD7・A8～A12 → GPIO26～38 に直結。/RD・/WR・A13 → U2(74LVC245)→ GPIO40～42
ミラーリング	GPIO43 → JP1 → C1RAM A10(18番)。JP2/JP3 で PPU A10/A11 に固定もできる
/IRQ	GPIO45 → R4 → Q1 → /IRQ(15番)
電源	本体 +5V(30・31番)→ D1 → Core2350B の VBUS → モジュール内 LDO → 3.3V → U2
本体 5V 検出	+5V → R2/R3 分圧 → GPIO46
音声	45番 → R7(0Ω)→ 46番。GPIO44 の PWM → R9/C3 → VR1(音量)→ R8 → 46番。46番 → C4 → R10/R11 → J2(3.5mm ライン出力)
PC	Core2350B の USB(付属の FPC-USB アダプター)。同じ D+/D- を J6(予備ランド)にも出している
テスト	J4(カセット60ピン全部、端子の真上)、J5(内部信号)、J3(SWD)、TP1～TP5

3.2 主要部品

記号	部品	役割
U1(P1～P4)	Waveshare Core2350B2	RP2350B(QFN-80、GPIO 48本)、16MB フラッシュ、2MB PSRAM、LDO(ME6217C33)、12MHz 水晶。25.4mm 角、外周2列 64ピン
U2	SN74LVC245AN(DIP-20)	PPU の /RD・/WR・A13 を 5V → 3.3V。5V 非トレラントの GPIO40～42 へ渡す
Q1	2SC1815(TO-92)	/IRQ をオープンコレクタで引き下げる
D1	1N4001～1N4007(ショットキー 1A 推奨)	本体 5V → モジュールの VBUS。USB 給電時の本体への逆流を防ぐ
J1	基板エッジ(金メッキ端子 60本)	ファミコンのカセットスロット
J2	PJ-324M	3.5mm ライン出力(モノラルを L/R 両方へ)
VR1	KOA KVSF637AC104(100kΩ、上面調整)	拡張音源の音量。足は ①③ が 5.0mm 間隔、② がその中央の 5.0mm 上(データシートの推奨取付穴、φ1.2)

3.3 電源

項目	値
供給元	本体の +5V(カセット端子 30・31番)。GND は 1・16番
モジュールへの経路	+5V → D1 → VBUS_MOD(モジュール P2-15 VBUS)→ モジュール内 LDO → +3V3
VBUS_MOD の電圧	1N400x で約 4.3V、ショットキーなら約 4.6V(LDO の入力として十分)
+3V3 の使い道	モジュールの 3V3 出力(P2-16)を U2 の電源と DIR に使う。C2 100nF をバスコンに
平滑	C1 10μF(VBUS_MOD)
USB 給電	Core2350B の USB から VBUS に入る。D1 があるので本体側の +5V へは流れない
消費電流(見込み)	モジュール数十 mA ～ 100mA 程度(RP2350B のクロック次第)。本体の 7805 の範囲内

逆流に注意:USB だけで給電している時(本体の電源が切れている時)に RP2350B がバスへ出力すると、電源の入っていない本体の IC に 3.3V が流れ込む。これを避けるため、本体の 5V を分圧して GPIO46 で監視する(3.6)。

3.4 バスの接続と電圧

CPU バス(26本、すべて RP2350B に直結)

信号	GPIO	向き
CPU D0～D7	0～7	双方向(読み出し時だけ RP2350B が出力)
CPU A0～A14	8～22(A0 から連続)	入力
/ROMSEL	23	入力(= !(A15 & M2))
R/W	24	入力
M2	25	入力

PPU バス

信号	GPIO	向き・経路
PPU AD0～AD7(カセット端子の PPU D0～D7)	26～33	双方向。/RD が下がる直前はアドレス下位 8 ビット、その後データ
PPU A8～A12	34～38	入力。GPIO26～38 で 13 ビットのアドレスが連続する
PPU /RD	40	U2 経由の入力
PPU /WR	41	U2 経由の入力
PPU A13	42	U2 経由の入力(カセット端子 56番、非反転)
PPU /A13	—	CIRAM /CE(48番)へ R1(0Ω)で直結。RP2350B にはつながらない

信号	GPIO	向き・経路
PPU A0～A7(ラッチ後)	—	RP2350B では使わない。J4 には出している

電圧の扱い

GPIO	5V を受けられるか	この基板での使い方
0～38	可(I/OVDD に電源が入っている時)	5V のバス信号を直接受ける
39	可	モジュール上の LED(470Ω)が付いているので使わない
40～47(ADC 兼用)	不可	40～42 は U2(3.3V 電源の 74LVC245)の後。43～46 は出力専用か分圧後の入力。47 はモジュール上の PSRAM の CS

- RP2350B の出力は 3.3V。2A03・PPU(NMOS)の入力は H を 2.0～2.4V 程度から認識するので受けられる。
- U2:DIR = +3V3(A→B)、/OE = GND(常に有効)。A 側(5V)に /RD・/WR・A13、B 側(3.3V)から GPIO40～42。空き入力 A4～A8 は GND。74LVC は電源が無い時も入力に 5V を受けられる。
- PIO は1ブロックで連続 32本しか扱えないので、CPU 用は GPIO0～31、PPU 用は GPIO16～47 の窓を使う。

3.5 PPU アドレスの読み取り方

ファミコンの PPU は、アドレス下位 8 ビットとデータを同じ 8本(AD0～AD7)で時分割する。本体側の 74LS373 がアドレスをラッチし、カセットには PPU A0～A7 として出てくるが、この基板ではそれを使わない。

- PIO で「/RD が High の間、AD0～AD7 を読み続ける」ループを回し、/RD が下がった時点で最後に読んだ値を下位アドレスとする。
- 上位は A8～A12(と A13)を直接読む。
- PPU の読み出しは約 372ns ごと(PPU クロック 5.37MHz の 2 クロック)。/RD が下がってからデータを出すまでの余裕が足りるかは**未検証**(TP1・TP2 で測る)。
- 足りない場合でも、画面転送(FC-PICO 方式)・自作ゲーム・NSF には影響しない。市販の CHR-ROM ゲームの動作に影響する。

3.6 制御・周辺回路

本体 5V の検出

+5V → R2 15kΩ → FC_5V_SENSE(GPIO46)→ R3 22kΩ → GND。5V のとき 2.97V、4.75V のとき 2.82V。ファームは GPIO46 が High になるまで、全てのバスのピンを入力にしておく。

/IRQ

GPIO45(IRQ_DRV)→ R4 4.7kΩ → Q1 ベース(R5 100kΩ で GND に引き下げ)。Q1 コレクタ → /IRQ(15番)、エミッタ → GND。GPIO45 = High で /IRQ が Low(割り込み要求)。ベース電流は約 0.55mA。起動中は R5 で Q1 がオフ。R6(10kΩ、+5V へのプルアップ)は未実装。本体側にプルアップがあることを確認済み。

ミラーリング(CIRAM A10、18番)

JP1～JP3 のどれか1つだけをはんだで閉じる。

ジャンパー	18番につながるもの	用途
JP1	GPIO43(CIRAM_A10_MCU)	ファームで切り替える(MMC1/MMC3 などのマップパー)
JP2	PPU A10(53番)	垂直ミラー固定
JP3	PPU A11(54番)	水平ミラー固定

GPIO43 は 3.3V 出力。本体の VRAM が 3.3V を H と認識するかは実機で確認する。駄目なら JP2/JP3 を使う。

音声

部分	回路	値
素通し	45番(本体からの音声)→ R7 0Ω → 46番(本体へ戻す)	市販カセットと同じ
拡張音源	GPIO44 PWM → R9 1kΩ → EXP_AUDIO_RC(C3 10nF で GND)→ VR1 100kΩ(①=RC、③=GND、②=中点)→ R8 47kΩ → 46番	RC の遮断周波数 約 15.9kHz。VR1 で音量 0～最大。どこに回しても本体の音声ラインには 47kΩ 以上が付く
ライン出力	46番 → C4 10μF(+ 側が 46番)→ LOUT(R10 47kΩ で GND)→ R11 100Ω → J2	低域の遮断 約 0.34Hz。J2 は L/R に同じ信号

操作

部品	動作
SW1 BOOTSEL	押しながりリセットすると RP2350B が USB 書き込みモードになる
SW2 RESET	RP2350B の RUN を GND に落とす。本体のリセットではない(本体はリセットボタンで)

3.7 デバッグ・テスト用

記号	内容
J3	SWD(1=SWCLK、2=GND、3=SWDIO)。debugprobe の Pico をつなぐ
J4	カセット60ピン全部(2x30、2.54mm)。差し込み部のすぐ上で、各列が真下の端子と同じ信号。本体に入る高さなのでランドとして使う想定
J5	内部信号(2x8、2.54mm)。上辺。12～14番は GND(元の LOUT・Q1_B・LINE は2層に収めるために外した。J2・R10・R11・Q1 の足で測れる)
J6	USB の予備ランド(1x4、2.54mm、1=5V(VBUS)、2=D-、3=D+、4=GND)。モジュールの FPC と同じ USB 信号。FPC 側と同時に機器をつながない。VBUS は本体給電時 D1 の後の約 4.3V
TP1～TP5	/RD、PPU D0、M2、/ROMSEL、GND。オシロのプロブ用(穴 0.7mm)

3.8 機械仕様

項目	値
外形	90.0 × 65.8mm(本体 90.0 × 55.1mm + 差し込み部 78.4 × 10.7mm)。規格の基板(56.8mm)より 9mm 高い
厚み	1.2mm
端子	60本、2.54mm ピッチ。部品面(表)= 1～30番、裏 = 31～60番。部品面から見て左端が 1/31番
端子の寸法	信号 1.6 × 5.7mm。1・30・31番は 2.6mm 幅、1・16・30・31番は差し込み側へ 1mm 長い(先に接触)
向き	部品面を本体の手前に向けて挿す(市販カセットのラベル面と同じ側に 1～30番)
レジスト	端子部は抜いてある(端子の上端の 0.6mm 上から先端まで)
部品	部品面。ただし R1 だけ裏面(Q1 を下へ倒して寝かせるため、2026-09-26 に移した)。スルーホール部品のみ(表面実装はソルダージャンパーと端子だけ)
シルク	表面: ロゴ(幅 19.5mm)と「rev0.2」。裏面: ロゴ(34mm)・「MADE BY SHIMA-R7」・スペック表(モジュールのソケットの内側)
モジュールの高さ	ピンソケットで載せるので、基板から約 13mm
コンデンサー	C1～C4 は足を曲げて右へ寝かせる前提。倒した本体の範囲は部品禁止(シルクに枠と「lay →」)
J2	3.5mm ジャックの差し込み口が左辺から約 3.6mm 外へ出る
ケース	printables 860420 のシェル(CC BY 4.0)を改造して、Bambu Lab P1S で 3D プリントする(2026-09-27 時点でユーザーが編集集中)

3.9 基板の製造仕様

項目	値
層数	2(FCu / B.Cu)。GND ベタを両面に、GND スティッチングビアあり

項目	値
配線幅	信号 0.25mm(IC の足の間で最小 0.15mm)。+5V 0.8mm(端子31番から J4 への枝だけ 0.5mm)、VBUS_MOD 0.6mm、GND 0.5mm、+3V3 0.4mm、音声 0.4mm
クリアランス	0.2mm
ビア	径 0.6mm / 穴 0.3mm
端子部	端子の隙間・両脇・先端は配線・ビア・ベタ禁止。配線はパッドの上端から入る
DRC	エラー 0、未接続 0(2026-09-26)。フットプリントをライブラリから書き換えた旨の通知 9件のみ
発注	JLPCB(2026-09-27 の時点で発注済み・未着)。厚み 1.2mm、黒、HASL(有鉛)、Gold Fingers、差し込み側 45° 面取り、5枚

3.10 ファームウェアが守ること

1. **起動時は全バスピンを入力**にし、GPIO46(本体 5V 検出)が High になってから出力を許す。
2. CPU D0～D7 を出力するのは、読み出しサイクル(/ROMSEL = Low かつ R/W = High、または \$6000～\$7FFF の WRAM 読み出し)の間だけ。
3. PPU AD0～AD7 を出力するのは、/RD = Low かつ A13 = Low(パターンテーブル)の間だけ。A13 = High(ネームテーブル)の時は本体の VRAM が出力する。
4. RP2350 のエラッタ E9:バスのピンで内部プルダウンを使わない。
5. GPIO39(LED)・GPIO47(PSRAM CS)をバスに使わない。
6. /IRQ は GPIO45 = High で要求、Low で解除。
7. 電源投入直後は本体の CPU が RP2350B の起動を待たずに動く。間に合わなければ本体のリセットボタンを押す。

3.11 モジュール単体の試験で使ったピン(2026-09-27)

基板が届く前に、Core2350B2 にピンヘッダーを付けて試した時のつなぎ方。ピンの番号は kicad/gen_schematic.py の並び(P1～P4 は 1 番が GND)。

モジュールのピン	信号	つないだ先	目的
P2-7 / P2-9	USB D- / D+	ESP32-C6 の GPIO12 / GPIO13	RP2350 を USB ホストにして C6 と直結
P2-15 / P2-14	VBUS / GND	C6 ボードの 5V / GND	2 枚とも C6 の UART の USB-C から給電
P2-8	GND	C6 の GND	データ線の横の GND
P2-10 / P2-6	SWCLK / SWDIO	debugprobe(Pico)の GP2 / GP3	SWD で書き込み・停止・メモリーの読み出し
P1-2	GPIO0(UART0 TX)	debugprobe の GP5	host_test のログ(届かず、未解決)
P2-4 → P2-16	GPIO46(本体 5V 検出)→ 3V3	線でつなぐ	ファミコンが無くても「5V あり」にして画面転送のループを動かす
P4-15	GPIO44(音の PWM)	1kΩ → 10μF → イヤホン(根元は P4-1 GND)	PC の音を耳で確かめる

4. ファームウェア(RP2350)

4.1 ファームウェアの一覧

pwsh -File firmware/build.ps1 で全部作る(firmware/out/)。大きさは今のビルド。

ファイル	大きさ	用途	中身
bus_test.uf2	93KB	最初の動作確認 1	CPU バスの読み書き。テスト ROM が 440Hz を鳴らし、\$8000 への書き込みを数えて LED を点滅

ファイル	大きさ	用途	中身
chr_test.uf2	94KB	最初の動作確認 2	PPU の読み出しに CHR を返す。4 色の縦縞と 8 ドットの格子が出れば下位アドレスの読み取りが正しい
host_test.uf2	72KB	試験	USB ホストで ESP32-C6 から受ける速さを測る(ファミコン無し)。UART0(GPIO0)にログ
magicon.uf2	201KB	ふだん使う	ゲーム・NSF・画面転送・メニュー。USB はデバイス(PC 直結、USB シリアル、picotool の自動書き込み)
magicon_wifi.uf2	211KB	Wi-Fi の画面転送	magicon と同じ。USB をホストにして J6 の ESP32-C6 とやりとりする。USB シリアル無し
usb_test.uf2	44KB	試験	USB 直結の速さを測る(ファミコン無し)。受けた画面を捨て、FCIN を 60 回/秒返す

4.2 作りの全体

項目	内容
クロック	250MHz(コア電圧 1.20V に上げてから)
コア 1	バスの応答だけ。割り込み無しの 1 本のループを SRAM で動かす(__not_in_flash_func)。モードごとに bus_loop / nsf_bus_loop / remote_bus_loop
コア 0	起動、ROM の読み込み、USB(シリアル・画面転送・音)、本体の電源の見張り、メニューで選ばれた番号の見張り
PIO0 SM0	cpu_m2.pio(GPIO 0～31 の窓)。CPU の全サイクル
PIO1 SM0	ppu_chr.pio(GPIO 16～47 の窓)。PPU の /RD
PIO2 SM0	mirror.pio(GPIO43)。PPU A10 / A11 を CIRAM A10 へ写す
DMA	音(PWM へ 32kHz で流す)に 1 チャンネルと DMA タイマー 1 つ
SRAM の主な使い道	ROM 384KB(PRG の後ろに CHR)、CHR-RAM 8KB、WRAM 8KB、NSF ドライバー 4KB、音のリング 16KB
読み出しの作り	8KB(PRG)/ 1KB(CHR)ごとのポインターの表を引くだけ。マップの計算は書き込みの時だけする

4.3 起動の流れ

1. バスのピン(GPIO 0～46、LED の 39 を除く)を全部入力・プル無しにする(RP2350 のエラッタ E9 のため、バスにプルダウンを使わない)。
2. コア電圧を 1.20V にして 250MHz にする。USB を始める(magicon はデバイス + USB シリアル、magicon_wifi はホスト)。
3. フラッシュの ROM 置き場を読む(FCMG = 1 本、FCLB = ライブラリ)。ライブラリなら、ウォッチドッグのメモ(scratch[0])にメニューで選んだ番号が残っていればその ROM、無ければメニューを組み立てる。1 本だけのライブラリはそのまま読む。
4. ROM を SRAM に写す(本体がリセットを解く前にバスへ出られるよう、5V を待つ前に済ませる)。駄目なら試験画面にする。
5. 本体の 5V(GPIO46)が来るまで待つ(LED を点滅、USB シリアルに waiting for Famicom 5V)。
6. PIO を設定し、コア 1 でバスの応答を始める(cart_start)。
7. コア 0 はモードごとのループへ。画面転送なら音を始めてから画面転送のループ。

4.4 CPU バスの応答

項目	内容
起こし方	M2 で起きて全サイクルを拾う(cpu_m2.pio)。M2 が上がって 80ns 後にアドレス・R/W・ROMSEL を読む
読み出し	\$8000-\$FFFF(/ROMSEL = Low)は PRG、\$6000-\$7FFF は WRAM(マップ 87 / 184 以外)。それ以外は出さない
書き込み	PIO は M2 が下がる所でデータをもう一度読んで 2 回目を送る。コア 1 がマップ・WRAM・PPU レジスターの横読みへ振り分ける
出す時	D0～D7 を出すのは読み出しサイクルの間だけ(3.10 の決まり)
数えるもの	CPU サイクル(約 179 万/秒)・PPU 読み出し・書き込みの回数(USB シリアルに 1 秒ごと)

4.5 PPU バスの応答

項目	内容
アドレス	/RD が High の間 AD0～AD7 を読み続け、/RD が下がった時に最後に読んだ値を下位 8 ビットとする。上位は A8～A12
出す時	/RD = Low かつ A13 = Low(パターンテーブル)の時だけ。ネームテーブルは本体の VRAM が出す
CHR-RAM	PPU の書き込みではアドレスが読めないで、CPU の \$2006 / \$2007 への書き込みを横から読んで同じ所へ書く。\$2000 の +1 / +32、\$2002 の読み出し・\$2005 で切り替わる「2 回目か」も真似る
MMC3 の IRQ	PPU A12 の立ち上がりを数え(A12 が Low の読み出しが 8 回続いた後)、GPIO45 → Q1 で /IRQ を出す
ミラーリング	PIO2 が PPU A10 / A11 を CIRAM A10(GPIO43)へ写す。水平・垂直・1 画面。JP1 を閉じた時だけ効く
未検証	/RD が下がってからデータを出すまでに間に合うか(chr_test と TP1・TP2 で測る)。駄目でも画面転送・NSF・CHR-RAM のゲームには響かない

4.6 マッパー

PRG + CHR の合計 384KB まで(SMB3 の 256K + 128K が入る)。tools/nes_pack.py の表から作った。

番号	マッパー	特徴
0	NROM	バンク切り替え無し
1	MMC1	CHR-RAM も可。ミラーリング切り替え(JP1)
2	UxROM	CHR-RAM
3	CNROM	CHR 8KB 切り替え
4	MMC3	スキャンライン割り込み(/IRQ)。ミラーリング切り替え(JP1)
7	AxROM	1 画面ミラー(JP1)
87	Jaleco J87	\$6000-\$7FFF の書き込みで CHR 切り替え
184	Sunsoft-1	\$6000-\$7FFF の書き込みで CHR 4KB × 2

- WRAM(\$6000-\$7FFF)8KB は電源を切ると消える(バッテリーのセーブはまだ)。
- 対応していない ROM は nes_pack.py がその場で止める(ライブラリなら飛ばす)。

4.7 NSF プレイヤー

項目	内容
置き場	曲のデータ = \$8000-\$FFFF(4KB × 8、\$5FF8-\$5FFF でバンク切り替え、384KB まで)。ドライバー = \$5000-\$5FFF(SRAM、本体側に何も無い所)
ベクター	\$FFFA-\$FFFF の読み出しだけ、ドライバーの番地を返す
曲を始める時	APU を黙らせ、本体 RAM と \$6000-\$7FFF を 0 にし、バンクを初期値に戻して INIT を呼ぶ(NSF の決まり)
再生	VBlank の NMI(約 60.1Hz)ごとに PLAY。再生間隔が標準と違う NSF はテンポが変わる
操作	←→ = 前 / 次の曲、A = 最初から、START = 一時停止
画面	ロゴ・曲名・作者・著作・曲番号。フォントは Consolas Bold 9pt から作った(gen_nsf_driver.py)
変数	\$5F80-(NSF がゼロページを自由に使うため)
未対応	拡張音源(VRC6・FDS・N163 など)、load が \$8000 より下のバンク無し NSF、FDS 用の \$6000-\$DFFF を RAM にする NSF

ドライバーの番地(nsf_driver.h から):

名前	値	意味
NSFD_BASE	0x5000	ドライバーの先頭(\$5000-\$5FFF はカセットの SRAM)

名前	値	意味
NSFD_NT	0x0800	画面の土台(ドライバーの先頭から)
NSFD_PARAM	0x0F00	ファームが書く値: 曲数・最初の曲・バンクの初期値(先頭から)
NSFD_RESET	0x5000	リセットベクターが指す番地
NSFD_NMI	0x519A	NMI(PLAY を呼び、パッドを読む)
NSFD_IRQ	0x5288	IRQ(何もしない)
NSFD_INIT_OPERAND	0x028A	INIT への JMP の番地の置き場(ファームが書く)
NSFD_PLAY_OPERAND	0x028D	PLAY への JMP の番地の置き場

4.8 画面転送(リモートデスクトップ)モード

項目	内容
絵	256 x 240 ドット、2 ビット(4 色)。16 x 16 ドットごとに 4 組のパレットから選ぶ(背景の 1 色は共通)
しくみ	ネームテーブルに「(行 % 8) * 32 + 列」のタイルを並べておき、PPU が読みに来たタイル番号と行の中の y から画面の位置を割り出して、その 8 ドットを返す。8 行ごとの組は、タイル行が 7 → 0 になる回数で数える
2 枚持ち	絵と付随(パレット 32・属性 64・OAM 256)を表と裏で 2 組。1 枚届いたら、画面の最後(行 29 → 0)で入れ替える。表示待ちがある間は次を読まない(PC 側がそこで待つ)
置き場	絵は ROM の SRAM(rom[])の先頭を使う
カーソル	スプライト 4 タイル(16 x 16、パターンテーブル \$1000)。PC が位置と表示の有無を送る
6502 の仕事	VBlank ごとに OAM DMA、パレット 32 と属性 64 を PPU へ、v を \$2000 に戻す。1コンとキーボード(9 行 x 2 列、1 列ごとに 500μs 待つ)を読んで \$5F00- に書く
レジスター	\$2000 = \$88(NMI 有効、背景 \$0000、スプライト \$1000)、\$2001 = \$1E

6502 側の番地(remote_driver.h から):

名前	値	意味
RD_PAL	0x5D00	パレット 32 バイト(VBlank ごとに 6502 が PPU へ写す)
RD_ATTR	0x5D20	属性テーブル 64 バイト
RD_OAM	0x5E00	スプライト(カーソル)256 バイト。OAM DMA の元
RD_IO	0x5F00	6502 が書く入力: +0 パッド、+1~+9 キーボード、+15 フレームの数

4.9 PC の音

項目	内容
出口	GPIO44 の PWM → R9 1kΩ・C3(RC 約 15.9kHz)→ VR1 → R8 47kΩ → カセット端子 46 番(本体の音声に混ざる)。J2(3.5mm)にも出る
PWM	10 ビット(0~1023、無音 = 512)、250MHz / 1024 = 約 244kHz
流し方	8192 サンプル(256ms)のリング(16KB 境界)を、DMA が「終わり無し」+ 読み出しリングで比較値へ流し続ける。間隔は DMA タイマー (250MHz × 4 / 31250 = 32kHz)
溜め方	届いた音は、鳴らしている所の 100ms 先に置く。150ms 以上溜まったら捨てる。鳴らし終えた所は無音で埋める(途切れても古い音を繰り返さない)
形式	32kHz モノラル。16 ビット PCM / 8 ビット PCM / IMA ADPCM(4 ビット、パケットごとに最初の値と段を載せる)(7.4)
使う時	画面転送モードの時だけ

4.10 ゲーム選択メニュー

項目	内容
形	NROM(PRG 32KB + CHR 8KB)。ファームが起動時にその場で組み立てる(cart_load_menu)。プログラムは gen_menu_driver.py、フォントとロゴは NSF プレイヤーの CHR
画面	ロゴ、「SELECT A GAME」、20 行の一覧(題名 28 文字)、操作の説明
操作	↑ ↓ = 選ぶ(24 フレーム押し続けると 4 フレームごとに動く)、← → = 20 項目ずつ、A / START = 始める。1コンと拡張端子のコントローラーの両方を読む
始め方	1) 本体 RAM \$0300 に写した 57 バイトのプログラムへ移る 2) NMI・表示・音を止め、選んだ番号を \$5FF0 に書く 3) VBlank を 90 回(1.5 秒)待つ 4) 本体 RAM を 0 にして JMP (\$FFFC)
カセット側	\$5FF0 への書き込みをコア 1 が見て、コア 0 が「0xFC01 + 番号」を scratch[0] に書いてウォッチドッグで再起動。次の起動でその ROM を読む(4.3)
戻り方	本体の電源を入れ直すと、メニューから(4.11)。本体のリセットボタンは今のゲームの最初に戻るだけ

メニューの番地(menu_driver.h から):

名前	値	意味
MENUD_COUNT	0x0000	項目の数(PRG の先頭から)
MENUD_PTR_LO	0x0100	題名の番地の下位の表
MENUD_PTR_HI	0x0180	同じく上位
MENUD_TITLES	0x0200	題名(28 文字ずつ)
MENUD_NT	0x1800	画面の土台 1024 バイト
MENUD_CODE	0x4000	メニューのプログラム(\$C000)
MENUD_SELECT	0x5FF0	選んだ番号を 6502 が書く番地(ファームが見て再起動)
MENUD_MAX_ITEMS	0x007F	項目の上限

4.11 本体の電源の見張り

- ゲーム・NSF・メニュー・画面転送のどのループでも、コア 0 が GPIO46(本体 5V)を見ている。
- 50ms 続けて Low なら scratch[0] を 0 にしてウォッチドッグで再起動する。USB から電源をもらっている時も、本体の電源を入れ直せばメニューに戻る。
- 再起動した後は 4.3 のとおり、5V が来るまでバスに出さない。

4.12 USB

項目	magicon(デバイス)	magicon_wifi(ホスト)
TinyUSB	pico-sdk 付属 0.18	0.21.0(C:\Users\Yugo\pico\tinyusb-0.21.0)
相手	PC(USB シリアル = CDC)	J6 の ESP32-C6(USB Serial/JTAG = CDC ACM)
USB シリアル(printf)	あり	無し
picotool の自動書き込み	あり(load_rom.ps1 がそのまま使える)	無し(BOOTSEL + RESET)
画面転送の読み書き	stdio_get_until / stdio_put_string	tuh_cdc_read / tuh_cdc_write(入り切らない入力捨てて次のフレームで送る)

magicon_wifi の注意:

- pico-sdk の USB シリアル(stdio_usb)は、アプリが TinyUSB のホストを使うと丸ごと無効になる。そのため 1 つのファームウェアにまていない。
- TinyUSB 0.18 の RP2350 ホストは、バルク転送が「割り込み用の窓口」で 1ms に 64 バイト = 64KB/秒止まりで、返事に受け取ったデータが混ざって化けた。0.21 は EPX を順番に回す。

- 0.21 と pico-sdk 2.3.1 の組み合わせでは CDC の受信 FIFO の mutex が初期化されず止まったので、CFG_TUSB_OS = OPT_OS_NONE(と tusb_time_millis_api)にした。
- C6 を開く時は DTR だけ立てる(C6 の USB Serial/JTAG は RTS でリセットがかかる)。受信バッファ 4096 バイト。

4.13 LED と表示

場面	LED(GPIO39)	USB シリアル(magicon)
5V を待つ間	100ms ごとに反転	waiting for Famicom 5V (GPIO46)... ROM: ok など
動いている間	点灯	1 秒ごとにモード・マップ・大きさと、CPU サイクル・PPU 読み出し・書き込みの回数
画面転送	1 枚受け取るごとに反転	—

LED は SIO のレジスターを直接書く(host_test で gpio_xor_mask64 が NOCP の HardFault を起こしたため。magicon では起きていないが同じ書き方にそろえた。原因は未説明)。

5. Wi-Fi ブリッジ(ESP32-C6)

項目	内容
ボード	ESP32-C6 N4(フラッシュ 4MB)。USB-C が 2 つ: チップの USB(GPIO12 = D-, GPIO13 = D+)と UART(CH343 経由)
ファームウェア	firmware/wifi_bridge/wifi_bridge.ino(Arduino-ESP32 3.3.12)。build_bridge.ps1 でビルドと書き込み
動き	TCP 5000 番と USB(Serial/JTAG)の間でバイトを素通しする。中身は見ない。USB へ書けない間は TCP から読まない(PC 側が待つ)
つながる前	USB に 2 秒ごとに fc-magicon wifi_bridge ip=... を出す(カセットは FCFR を探して読むので邪魔にならない)。USB から来た物は捨てる
名前	mDNS で fc-magicon.local(サービス _fcbridge._tcp)。remote_pc.py --tcp は既定でこの名前につなぐ
Wi-Fi	2.4GHz。SSID とパスワードは wifi_secrets.h(wifi_secrets.example.h を写して書く。GitHub には上げない)。省電力は切る
様子	UART0(UART の USB-C)に 1 秒ごと: Wi-Fi の状態・IP・TCP の有無・USB の有無・行き来したバイト数
フラッシュの癖	QIO / 80MHz だと読み出しが化けて起動しない。ブートローダーは DIO / 40MHz(FQBN の FlashMode=dio, FlashFreq=40)、アプリはコンストラクターで SPI1 を 40MHz に落とす
電源の癖	弱めで、一度ブラウンアウトでリセットを繰り返した(USB の挿し直しで直った)
カセットとのつながり方	J6 の 2 番(D-)→ GPIO12、3 番(D+)→ GPIO13、4 番 → GND。C6 の電源は別を取る(J6 の VBUS から取ると Wi-Fi の電流が本体の 5V に乗る)。J6 と FPC の USB に同時に機器をつながない

6. PC のソフトウェア

使うのは Python 3(Pillow・numpy)と Windows の API(ctypes)。pip で足すものは無い(test_nsf_driver.py の py65 と、書類の pypdf だけ別)。

6.1 道具の一覧

道具	役目
firmware/tools/remote_pc.py	画面転送の PC 側。画面を取り込んで送り、入力をマウス・キーへ、音も送る(6.2)
firmware/tools/nesframe.py	PC の画像を画面転送の 1 枚にする(6.3)
firmware/tools/pcaudio.py	PC の音の横取りと、送る形式への変換(6.4)
firmware/load_rom.ps1	ROM・NSF・画面転送モード・ライブラリをフラッシュに書く(picotool)
firmware/tools/nes_pack.py	ROM を調べて書く形にする。ライブラリを作る
firmware/tools/asm6502.py	6502 の小さなアセンブラ(Python の関数で書く)。試験 ROM・ドライバー・メニューに使う

道具	役目
firmware/tools/usb_bench.py / wifi_bench.py / host_bench.py / audio_bench.py	速さ・音の試験(8 章)
firmware/tools/test_keyboard_bridge.py	キーボードの割り当て・トグル・チャタリング対策の試験
firmware/sim/	PC の試験台(8 章)
firmware/build.ps1, wifi_bridge/build_bridge.ps1	ファームウェアのビルド
docs/build_*.py	仕様書・手順書を作る
kicad/*.py	回路図・基板・ガーバー・部品表・印刷物を作る
3d/export_step.py, case/label/make_label.py	STEP の書き出し、シールの画像と印刷

6.2 remote_pc.py

項目	内容
取り込み	GDI の StretchBlt で 256 x 240 に縮めて取り込む。表示は「画面全体を縮小」(縦横比を保ち上下は黒)と「カーソルのまわりを等倍」
変換	nesframe.encode(灰色 4 階調 / カラー)。カラーは前の画面の組から探す(チラつき防止)
送信	送受信は別スレッド(IoThread)。画面を作る間に前の 1 枚を送る(順番にやると 15 枚/秒しか出なかった)。音の packets は画面より先に送る
つなぎ先	USB シリアル(COM、Win32 の CreateFile)か TCP(--tcp)
マウス	十字キーで動かす(押し続けると 1 + 0.25 ずつ速く、12 まで。等倍表示では PC の 1 ドット単位)。A = 左クリック、B = 右クリック
表示の切り替え	SELECT = 縮小 ⇄ 等倍、START = 等倍の時にカーソルを追うか止めるか
キーボード	SendInput でスキャンコード(JIS 配列)を送る。6.6
安全	入力が 0.5 秒届かない時と止めた時は、押しっぱなしのキーとボタンを離す

オプション(remote_pc.py の build_parser() から):

オプション	内容
--port COMx	COM ポートを指定する(既定: USB シリアルを探す)
--color	カラー(既定は灰色 4 階調。文字が読みやすい)
--tcp HOST	Wi-Fi ブリッジ(ESP32-C6)経由でつなぐ。HOST を省くと fc-magicon.local
--fps	送る速さの上限(既定: USB 30、Wi-Fi 25。Wi-Fi は 25 までなら遅れ 40ms 前後)
--dry-run N	カセット無しで N 枚取り込んで変換し、速さを表示する
--no-audio	PC の音を送らない
--audio {pcm16 / pcm8 / adpcm}	音の形式(既定: USB 直結 pcm16 = 64KB/秒、Wi-Fi pcm8 = 32KB/秒。adpcm = 16KB/秒は雑音が多い)
--volume	送る音の大きさ(1.0 = そのまま)

6.3 画面の変換(nesframe.py)

項目	内容
灰色	全体で黒・濃い灰・薄い灰・白の 4 階調。文字がいちばん読みやすい
カラー	背景の 1 色(いちばん多い色、上下の黒い帯は数えない)+ 3 色 x 4 組のパレット。16 x 16 ドットごとに 4 組のどれかを選ぶ
組の決め方	組の区画でよく使われる色を候補に、3 つの枠を 1 つずつ入れ替えて誤差が減る時だけ採る
まだら対策	上下左右の区画と違う組には罰点(SMOOTH = 40)を足す。境目の段差 1.45 → 1.26(2026-09-27)
ディザ	Bayer 4 x 4 で 4 色の中間を点の混ぜ方で出す

項目	内容
色の近さ	CIELAB の距離(RGB だと暗い緑が茶色に寄る)。表を前もって作って速くする
ファミコンの色	agnes の表(PC の試験台の画面とドット単位で比べられるように)

6.4 PC の音(pcaudio.py)

項目	内容
録り方	WASAPI のループバックで既定の再生デバイスの音を録る(ctypes で COM を呼ぶ)。共有モード、溜め 200ms、20ms ごとに読む
変換	各チャンネルを平均してモノラル、3 点の移動平均で高い音を少し落としてから、直線補間で 32kHz に
形式	pcm16(64KB/秒)/ pcm8(32KB/秒)/ adpcm(IMA、16KB/秒)。既定は USB 直結 pcm16、Wi-Fi pcm8
聞き比べ(2026-09-27)	ADPCM は「サー」という雑音、16 ビットがいちばん綺麗、8 ビットは少し震えるが ADPCM より良い。「音より画面重視」で Wi-Fi は 8 ビット
試験	python pcaudio.py --test: ADPCM の往復 SN 比 32dB。1kHz を鳴らしてループバックで 1001Hz として録れる

6.5 ROM の書き込み

書き方	中身
load_rom.ps1 game.nes / music.nsf	1 本(FCMG)。マップパー・大きさを表示して、動くものだけ書く
load_rom.ps1 -Remote	画面転送モードの印(FCRD)を 1 本として書く
load_rom.ps1 -Library フォルダーやファイル [-Remote]	ライブラリ(FCLB)。題名順。題名はファイル名(大文字)、NSF は曲集の題名、titles.txt(ファイル名=題名)で変えられる
書き込み	picotool で -o 0x10800000。magicon が動いていれば自動で書き込みモードにして、書いた後に戻す(-f)

6.6 キーボードの割り当て

前に作った拡張端子 ⇔ USB 変換器(Famicom-Expand-USB-Adapter、実機で確認済み)と同じ。SHIFT・GRPH(Alt)・CTR はトグル(RETURN で SHIFT は戻る)、1 キーずつのチャタリング対策(2 回続けて同じなら確定)。記号は JIS 配列の位置で送る。

英字・数字(A〜Z、0〜9)は刻印どおり。それ以外:

ファミコンのキー	PC で押されるキー	ファミコンのキー	PC で押されるキー
F1	F1	↓	↓
F2	F2	ESC	Esc
F3	F3	GRPH	Alt(トグル)
F4	F4	INS	Insert
F5	F5	カナ	半角/全角
F6	F6	←	←
F7	F7	SHIFT(左)	Shift(トグル)
F8	F8	RETURN	Enter
,	,	→	→
-	-	SHIFT(右)	Shift(トグル)
.	.	SPACE	Space
/	/	STOP	BackSpace
:	:	↑	↑
;	;	¥	¥
@	@	[	[

ファミコンのキー	PC で押されるキー	ファミコンのキー	PC で押されるキー
CLR HOME	Home	]	]
CTR	Ctrl(トグル)	^	^
DEL	BackSpace	-	ろ(バックスラッシュ)

7. データと通信の形式

数はリトルエンディアン。

7.1 フラッシュの使い方

番地(フラッシュの先頭から)	大きさ	中身
0x000000	8MB	ファームウェア(uf2 で書く。magicon は約 200KB)
0x800000	8MB	ROM 置き場: 1 本(FCMG)かライブラリ(FCLB、127 本まで)

- XIP の番地では 0x10800000。load_rom.ps1 は picotool で -o 0x10800000 に書く。
- 1 本の大きさは PRG + CHR で 384KB まで(RP2350 の SRAM に写して返すため)。ライブラリ全体は 8MB まで。
- ライブラリの項目は 64 バイト(題名 48 + 位置 4 + 長さ 4 + 合計 4 + 予備 4)、題名はメニューで 28 文字まで。

7.2 ROM 置き場: 1 本(FCMG)

位置	大きさ	中身
0	4	"FCMG"
4	4	中身の長さ
8	4	中身のバイトの合計(32 ビット)
12	4	予備
16	長さ	.nes / .nsf をそのまま、または "FCRD"(画面転送モード)

7.3 ROM 置き場: ライブラリ(FCLB)

位置	大きさ	中身
0	4	"FCLB"
4	4	項目の数(1~127)
8	4	全体の長さ
12	4	予備
16 + 64 × i	48	題名(ASCII、0 で終わる)
+48	4	中身の位置(置き場の先頭から)
+52	4	中身の長さ
+56	4	中身のバイトの合計
+60	4	予備
目次の後(16 バイト境界)	—	中身を並べる(1 本の時と同じ .nes / .nsf / "FCRD"、16 バイト境界)

7.4 画面転送の通信

USB でも TCP でも同じ並び。カセットは FCFR / FCAU を 1 バイトずつ探すので、途中から読み始めても合う。

パケット	向き	並び	大きさ
画面	PC → カセット	"FCFR" + 絵 15360 + パレット 32 + 属性 64 + カーソルの x・y・表示(3)	15463
音	PC → カセット	"FCAU" + 形式 1 + サンプル数 2(1~1024)+ 中身	7 + 中身
入力	カセット → PC	"FCIN" + 16(下の表)。6502 のフレームの数が変わるたび(約 60 回/秒)	20

音の形式	値	中身
16 ビット PCM	0	サンプル × 2 バイト(符号付き)
IMA ADPCM	1	最初の値 2 + 段 1 + 4 ビット × サンプル数(下位の 4 ビットが先)
8 ビット PCM	2	サンプル × 1 バイト(符号付き)

FCIN の位置	中身
0	パッド(bit7 = A、6 = B、5 = SELECT、4 = START、3 = ↑、2 = ↓、1 = ←、0 = →)
1~9	キーボード 9 行(下位 4 ビット = 列 0、上位 4 ビット = 列 1、押すと 0)
10~14	予備(試験用ファームでは受け取った枚数などを入れる)
15	6502 のフレームの数

7.5 画面のデータの並び

もの	並び
絵	タイル行 r(0~29)、行の中の y(0~7)、列 c(0~31)、面 p(0/1)の 1 バイト = fb[((r × 8 + y) × 32 + c) × 2 + p]。1 バイト = 横 8 ドットの 1 面
パレット	背景 16 + スプライト 16(ファミコンの色番号)
属性	ファミコンの属性テーブル 64 バイトそのまま(16 × 16 ドットごとの組)
カーソル	x(0~255)、y(0~239)、表示(0/1)

8. 試験と検証の道具

道具	何を確かめるか
sim/sim.exe(PC の試験台)	magicon/cart.c を PC でビルドし、エミュレーター agnes(MIT)のカセットとして動かす。CPU の全サイクルと PPU の読み出しを PIO と同じ形の値にして渡すので、実機と同じロジックを試せる(試せないのは PIO のナノ秒単位のタイミングだけ)。画面は BMP、音は WAV(sim/apu.c で 2A03 の音を作る)。ライブラリを渡すとメニューから始める
sim/run_compare.py	magicon と agnes 自身のマッパーで動かし、画面を毎フレーム比べる
sim/test_remote.py	画面転送: 届いた絵が画面にドット単位でそのまま出るか、カーソル、パッドとキーボードの読み取り
sim/make_test_nsf.py	バンク切り替えのある試験用 NSF(3 曲)
magicon/test_nsf_driver.py	NSF ドライバーを 6502 エミュレーター(py65)で動かす
tools/test_keyboard_bridge.py	キーボードの割り当て・トグル・チャタリング対策(キーは実際には送らない)
usb_test.uf2 + tools/usb_bench.py	USB 直結の速さ(ファミコン無し)
wifi_bridge + tools/wifi_bench.py	Wi-Fi → C6 → PC の速さ・遅れ・中身
host_test.uf2 + tools/host_bench.py	Wi-Fi → C6 → RP2350(USB ホスト)の速さと欠け
tools/audio_bench.py	カセットの音(ドレミを実際の速さで送る。形式を選ぶ)

道具	何を確かめるか
tools/pcaudio.py --test	ADPCM の往復とループバックの録音
debugprobe + OpenOCD	SWD で停止・メモリ・レジスターの読み出し、書き込み(set USE_CORE 0)。SWD からのリセットは起動がおかしくなるので、書いた後は電源を入れ直す

9. 性能(測った値)

すべて 2026-09-27、この PC。ファミコンの実機ではまだ測っていない。

項目	条件	結果
USB 直結の速さ	Core2350B2 単体 + usb_test、決まったデータを目いっぱい	614KB/秒 = 最大 40.6 枚/秒、150 枚すべて届く
画面転送(USB 直結)	remote_pc.py、usb_test 相手	灰色 29.8 枚/秒、カラー 20.0 枚/秒(PC 側で画面を作る速さで決まる)
PC 側で画面を作る速さ	remote_pc.py --dry-run	等倍 約 60 枚/秒、縮小 灰色 約 30 / カラー 約 23 枚/秒
Wi-Fi → C6 → PC	wifi_bench、目いっぱい	462KB/秒 = 30.6 枚/秒、食い違い 0
同上の遅れ	20～25 枚/秒で送る	中央 37ms、最大 73～81ms
画面転送(Wi-Fi、C6 まで)	remote_pc.py --tcp、PC 上の偽のカセット	灰色 24.5 枚/秒、カラー 19.9 枚/秒
Wi-Fi → C6 → RP2350(USB ホスト)	host_test、TinyUSB 0.21	420KB/秒 = 27.8 枚/秒、500 枚すべて届く
同上(TinyUSB 0.18)	host_test	64KB/秒止まり、返事のデータが化ける
音の溜まり具合	magicon_wifi + audio_bench、Wi-Fi	98～115ms で安定
マップの正しさ	sim、手元の 12 本(マップー 0/1/2/4 の 8 本を agnes と比べる)	1200 フレームすべて画面が一致

10. 開発環境

用途	道具	置き場・版
RP2350 のビルド	pico-sdk 2.3.1、xPack arm-none-eabi-gcc 15.2.1、CMake 4.4.3、Ninja 1.13.2、picotool / pioasm 2.3.1	C:\Users\Yugo\pico(日本語の無いパスへコピーしてビルド)
USB ホスト	TinyUSB 0.21.0	C:\Users\Yugo\pico\tinyusb-0.21.0(host_test・magicon_wifi だけ)
SWD	xPack OpenOCD 0.12.0-7、debugprobe v2.3.1(Raspberry Pi Pico)	C:\Users\Yugo\pico\openocd
PC の試験台	Zig 0.16.0(C コンパイラー)、agnes(MIT)	C:\Users\Yugo\pico\zig、firmware/sim/agnes
ESP32-C6	arduino-cli 1.5.1、Arduino-ESP32 3.3.12(C6 の分だけ)	C:\Users\Yugo\esp
基板	KiCad 10(pcbnew を Python から)、Freerouting 2.4.1	E:\KiCad
PC のソフト	Python 3.13、Pillow、numpy(書類に pypdf、NSF の試験に py65)	—
書類	Edge のヘッドレス、md2pdf.py(目次・ページ番号)	Claude\md2pdf.py
印刷	Brother DCP-J1270N	—
ケース	Bambu Lab P1S	—

11. 制限と未実装

項目	今	逃げ道・予定
バッテリーバックアップ	WRAM は電源を切ると消える	WRAM をフラッシュへ保存する
大きい ROM	PRG + CHR 384KB まで	PSRAM(2MB)を使う
マップ	8 種類	必要なものから足す
NSF の拡張音源	鳴らない	RP2350 で合成して GPIO44 の PWM へ(VRC6 → FDS → N163 → 5B → MMC5 → VRC7)。音の DMA の仕組みを使い回せる
PC 直結と Wi-Fi	ファームウェアが別(magicon / magicon_wifi)	USB シリアルとリセット用の窓口を自前で作れば 1 つにできる
magicon_wifi の書き換え	BOOTSEL + RESET が要る	Wi-Fi で ROM を送る
メニューの題名	ASCII、28 文字まで	かなのフォントを足す
Wi-Fi の音	8 ビット(少し震える)	ディザーでならず、または 16 ビットにして画面を少し落とす
host_test の UART ログ	debugprobe に届かない	配線を確認する
GPIO コプロセッサ	host_test で NOCP の HardFault(原因は未解明)	SIO を直接書いて避けている
SWD からのリセット	起動がおかしくなる	書いた後は電源を入れ直す

12. 確認状況

項目	状況
カセット端子の信号・電源・音声・IRQ・向き	実機・資料で確認済み(2026-09-26)
Core2350B2 のピン配置	照合シートで実物と全ピン一致
PIO の応答タイミング(CPU・PPU)	未検証 (基板が届いてから bus_test / chr_test で。TP1～TP4)
CIRAM A10 を 3.3V で動かす(JP1)	未検証 。駄目なら JP2 / JP3
マップ(0/1/2/3/4/7/87/184)	PC の試験台で確認済み(12 本。0/1/2/4 は agnes と全フレーム一致)
NSF プレイヤー	PC の試験台と py65 で確認済み
画面転送(絵・カーソル・パッド・キーボード)	PC の試験台で確認済み(ドット単位で一致)
キーボードの割り当て	test_keyboard_bridge.py で確認済み。割り当ては変換器で実機確認済み
ゲーム選択メニュー	PC の試験台で確認済み(37 項目、ページ送り、ゲーム・NSF・画面転送が始まる)。 実機では未検証 (1.5 秒の間に再起動が間に合うか)
ROM の書き込み(load_rom.ps1)	Core2350B2 単体で確認済み(mario3 384KB、-Remote)。-Library は引数の受け取りまで
USB 直結の速さ	Core2350B2 単体で確認済み(614KB/秒)
Wi-Fi(C6)→ USB	PC 相手・RP2350(USB ホスト)相手で確認済み
magicon_wifi	Core2350B2 単体で 1 枚目が裏画面に入るまで確認。 ファミコンでは未検証
PC の音	Core2350B2 単体 + Wi-Fi でイヤホンで確認。 ファミコンの音声(46 番)に混ぜては未検証 (VR1・R8 で合わせる)
本体の電源の見張り(5V が消えたら再起動)	未検証

13. 部品表

kicad/bom.csv から作った。

部品	数	記号	フットプリント	メモ
10μF 電解	1	C1	CP_Radial_D5.0mm_P2.00mm	16V以上、φ5 ピッチ2mm
100nF セラミック	1	C2	C_Disc_D3.0mm_W1.6mm_P2.50mm	U2 のパスコン
10nF セラミック	1	C3	C_Disc_D3.0mm_W1.6mm_P2.50mm	PWM の RC フィルタ
10μF 電解	1	C4	CP_Radial_D5.0mm_P2.00mm	φ5 ピッチ2mm。+側=SOUND_OUT(四角パッド)。ライン出力の直流カット。1μFフィルムでも可
1N4001～1N4007(ショットキー 1A があればそちら)	1	D1	D_DO-41_SOD81_P7.62mm_Horizontal	本体5V→モジュール。USB給電時の逆流防止。省略不可
PJ-324M	1	J2	PJ-324M	3.5mm ステレオジャック(SHVC-SOUND-ESP32 と同じ)
ピンヘッダー 1x3	1	J3	PinHeader_1x03_P2.54mm_Vertical	SWD(debugprobe の Pico 用)
ピンヘッダー or ピンソケット 2x30	1	J4	PinHeader_2x30_P2.54mm_Vertical	カセット60ピンのブレイクアウト。必要な時だけでもよい
ピンヘッダー 2x8	1	J5	PinHeader_2x08_P2.54mm_Vertical	デバッグ用(内部信号)
ピンヘッダー 1x4(任意)	1	J6	PinHeader_1x04_P2.54mm_Vertical	USB の予備ランド(5V/D-/D+/GND)。FPC 側と同時に機器をつながない
2SC1815	1	Q1	TO-92_Inline_Wide	TO-92 NPN。足は平らな面から E-C-B。他の小信号NPNでも可(足順に注意)。下(J4 側)へ倒して付ける
0Ω(ジャンパー線でも可)	1	R1	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	CIRAM /CE ← /A13。裏面に付ける(表は Q1 を倒す場所)。表の足先は短く切る
15kΩ 1/4W	1	R2	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	
22kΩ 1/4W	1	R3	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	
4.7kΩ 1/4W	1	R4	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	
100kΩ 1/4W	1	R5	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	
10kΩ(未実装)	1	R6	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	本体に /IRQ のプルアップが無い時だけ付ける
0Ω(ジャンパー線でも可)	1	R7	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	音声 45→46番の素通し
47kΩ	1	R8	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	拡張音源を46番に混ぜる抵抗(VR1 の中点から)
1kΩ 1/4W	1	R9	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	
47kΩ 1/4W	1	R10	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	
100Ω 1/4W	1	R11	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal	
タクトスイッチ 6mm	1	SW1	SW_PUSH_6mm	BOOTSEL
タクトスイッチ 6mm	1	SW2	SW_PUSH_6mm	RESET(RP2350のRUN)
74LVC245 DIP-20	1	U2	DIP-20_W7.62mm	SN74LVC245AN。5V入力を3.3Vへ。74HC/HCT245 では代用不可(5V入力に耐えない)
半固定抵抗 100kΩ KOA KVSF637AC104	1	VR1	KOA_SF6_637A	拡張音源の音量。上面調整、足 ①③ 5.0mm・② は 5.0mm 上
Waveshare Core2350B2	1	U1		RP2350B、2MB PSRAM。付属のFPC-USBアダプターで PC とつなぐ

部品	数	記号	フットプリント	メモ
ピンソケット 2x8(2.54mm)	4	P1-P4		Core2350B を載せる。モジュール側には 2x8 ピンヘッダー4個
ピンヘッダー 2x8(2.54mm)	4	(モジュール側)		Core2350B の裏に付ける
ICソケット DIP-20	1	(U2)		任意
PCB 90x65.8mm 2層 1.2mm	1			金メッキ端子(Gold fingers)、差し込み側 45°面取り

付録 A. ピン・接続の一覧

ネットリスト(kicad/FC-MAGICON.net)から kicad/gen_spec_tables.py が作った表。

カセット端子 J1(60ピン)

番	面	信号	つながる先
1	表(手前)	GND	GND(全部品共通)
2	表(手前)	CPU_A11	P3-5(GPIO19)
3	表(手前)	CPU_A10	P3-6(GPIO18)
4	表(手前)	CPU_A9	P3-3(GPIO17)
5	表(手前)	CPU_A8	P3-4(GPIO16)
6	表(手前)	CPU_A7	P3-2(GPIO15)
7	表(手前)	CPU_A6	P1-15(GPIO14)
8	表(手前)	CPU_A5	P1-16(GPIO13)
9	表(手前)	CPU_A4	P1-13(GPIO12)
10	表(手前)	CPU_A3	P1-14(GPIO11)
11	表(手前)	CPU_A2	P1-11(GPIO10)
12	表(手前)	CPU_A1	P1-12(GPIO9)
13	表(手前)	CPU_A0	P1-9(GPIO8)
14	表(手前)	CPU_R/W	P3-12(GPIO24)
15	表(手前)	/IRQ	Q1.2, R6.2
16	表(手前)	GND	GND(全部品共通)
17	表(手前)	PPU_/RD	TP1.1, U2.2
18	表(手前)	CIRAM_A10	J5.11, JP1.2, JP2.2, JP3.2
19	表(手前)	PPU_A6	—
20	表(手前)	PPU_A5	—
21	表(手前)	PPU_A4	—
22	表(手前)	PPU_A3	—
23	表(手前)	PPU_A2	—
24	表(手前)	PPU_A1	—
25	表(手前)	PPU_A0	—
26	表(手前)	PPU_D0	P3-14(GPIO26), TP2.1
27	表(手前)	PPU_D1	P3-13(GPIO27)
28	表(手前)	PPU_D2	P3-16(GPIO28)
29	表(手前)	PPU_D3	P3-15(GPIO29)

番	面	信号	つながる先
30	表(手前)	+5V	D1.2, R2.1, R6.1
31	裏(奥)	+5V	D1.2, R2.1, R6.1
32	裏(奥)	M2	P3-11(GPIO25), TP3.1
33	裏(奥)	CPU_A12	P3-8(GPIO20)
34	裏(奥)	CPU_A13	P3-7(GPIO21)
35	裏(奥)	CPU_A14	P3-10(GPIO22)
36	裏(奥)	CPU_D7	P1-10(GPIO7)
37	裏(奥)	CPU_D6	P1-7(GPIO6)
38	裏(奥)	CPU_D5	P1-8(GPIO5)
39	裏(奥)	CPU_D4	P1-5(GPIO4)
40	裏(奥)	CPU_D3	P1-6(GPIO3)
41	裏(奥)	CPU_D2	P1-3(GPIO2)
42	裏(奥)	CPU_D1	P1-4(GPIO1)
43	裏(奥)	CPU_D0	P1-2(GPIO0)
44	裏(奥)	/ROMSEL	P3-9(GPIO23), TP4.1
45	裏(奥)	SOUND_IN	R7.1
46	裏(奥)	SOUND_OUT	C4.1, R7.2, R8.2
47	裏(奥)	PPU_/WR	U2.3
48	裏(奥)	CIRAM_/CE	R1.2
49	裏(奥)	PPU_/A13	R1.1
50	裏(奥)	PPU_A7	—
51	裏(奥)	PPU_A8	P4-5(GPIO34)
52	裏(奥)	PPU_A9	P4-8(GPIO35)
53	裏(奥)	PPU_A10	JP2.1, P4-7(GPIO36)
54	裏(奥)	PPU_A11	JP3.1, P4-10(GPIO37)
55	裏(奥)	PPU_A12	P4-9(GPIO38)
56	裏(奥)	PPU_A13	U2.4
57	裏(奥)	PPU_D7	P4-6(GPIO33)
58	裏(奥)	PPU_D6	P4-3(GPIO32)
59	裏(奥)	PPU_D5	P4-4(GPIO31)
60	裏(奥)	PPU_D4	P4-2(GPIO30)

Core2350B のピン(ヘッダー P1～P4)

ヘッダー-ピン	モジュールのピン	信号	つながる先
P1-2	GPIO0	CPU_D0	J1.43
P1-4	GPIO1	CPU_D1	J1.42
P1-3	GPIO2	CPU_D2	J1.41
P1-6	GPIO3	CPU_D3	J1.40
P1-5	GPIO4	CPU_D4	J1.39
P1-8	GPIO5	CPU_D5	J1.38
P1-7	GPIO6	CPU_D6	J1.37

ヘッダー-ピン	モジュールのピン	信号	つながる先
P1-10	GPIO7	CPU_D7	J1.36
P1-9	GPIO8	CPU_A0	J1.13
P1-12	GPIO9	CPU_A1	J1.12
P1-11	GPIO10	CPU_A2	J1.11
P1-14	GPIO11	CPU_A3	J1.10
P1-13	GPIO12	CPU_A4	J1.9
P1-16	GPIO13	CPU_A5	J1.8
P1-15	GPIO14	CPU_A6	J1.7
P3-2	GPIO15	CPU_A7	J1.6
P3-4	GPIO16	CPU_A8	J1.5
P3-3	GPIO17	CPU_A9	J1.4
P3-6	GPIO18	CPU_A10	J1.3
P3-5	GPIO19	CPU_A11	J1.2
P3-8	GPIO20	CPU_A12	J1.33
P3-7	GPIO21	CPU_A13	J1.34
P3-10	GPIO22	CPU_A14	J1.35
P3-9	GPIO23	/ROMSEL	J1.44, TP4.1
P3-12	GPIO24	CPU_R/W	J1.14
P3-11	GPIO25	M2	J1.32, TP3.1
P3-14	GPIO26	PPU_D0	J1.26, TP2.1
P3-13	GPIO27	PPU_D1	J1.27
P3-16	GPIO28	PPU_D2	J1.28
P3-15	GPIO29	PPU_D3	J1.29
P4-2	GPIO30	PPU_D4	J1.60
P4-4	GPIO31	PPU_D5	J1.59
P4-3	GPIO32	PPU_D6	J1.58
P4-6	GPIO33	PPU_D7	J1.57
P4-5	GPIO34	PPU_A8	J1.51
P4-8	GPIO35	PPU_A9	J1.52
P4-7	GPIO36	PPU_A10	J1.53, JP2.1
P4-10	GPIO37	PPU_A11	J1.54, JP3.1
P4-9	GPIO38	PPU_A12	J1.55
P4-12	GPIO39	(未接続)	—
P4-11	GPIO40	PPU_/RD_3V3	U2.18
P4-14	GPIO41	PPU_/WR_3V3	U2.17
P4-13	GPIO42	PPU_A13_3V3	U2.16
P4-16	GPIO43	CIRAM_A10_MCU	JP1.1
P4-15	GPIO44	EXP_AUDIO_PWM	R9.1
P2-2	GPIO45	IRQ_DRV	R4.1
P2-4	GPIO46	FC_5V_SENSE	R2.2, R3.1
P2-3	GPIO47_PSRAM_CS	(未接続)	—
P1-1	GND	GND	GND(全部品共通)

ヘッダー-ピン	モジュールのピン	信号	つながる先
P2-1	GND	GND	GND(全部品共通)
P2-5	BOOTSEL	BOOTSEL	SW1.1
P2-6	SWD	SWD	J3.3
P2-7	USBD_N	USBD_N	J6.2
P2-8	GND	GND	GND(全部品共通)
P2-9	USBD_P	USBD_P	J6.3
P2-10	SWCLK	SWCLK	J3.1
P2-11	RUN	RUN	SW2.1
P2-12	ADC_VREF	(未接続)	—
P2-13	3V3_EN	(未接続)	—
P2-14	GND	GND	GND(全部品共通)
P2-15	VBUS	VBUS_MOD	C1.1, D1.1, J6.1
P2-16	3V3	+3V3	C2.1, U2.1, U2.20
P3-1	GND	GND	GND(全部品共通)
P4-1	GND	GND	GND(全部品共通)

J4 ブレイクアウト(2x30)

列 k(左から 1～30)の下の列 = カセット k 番、上の列 = カセット k+30 番。

列	下の列(ヘッダー番号)	カセット番・信号	上の列(ヘッダー番号)	カセット番・信号
1	1	1: GND	2	31: +5V
2	3	2: CPU_A11	4	32: M2
3	5	3: CPU_A10	6	33: CPU_A12
4	7	4: CPU_A9	8	34: CPU_A13
5	9	5: CPU_A8	10	35: CPU_A14
6	11	6: CPU_A7	12	36: CPU_D7
7	13	7: CPU_A6	14	37: CPU_D6
8	15	8: CPU_A5	16	38: CPU_D5
9	17	9: CPU_A4	18	39: CPU_D4
10	19	10: CPU_A3	20	40: CPU_D3
11	21	11: CPU_A2	22	41: CPU_D2
12	23	12: CPU_A1	24	42: CPU_D1
13	25	13: CPU_A0	26	43: CPU_D0
14	27	14: CPU_R/W	28	44: /ROMSEL
15	29	15: /IRQ	30	45: SOUND_IN
16	31	16: GND	32	46: SOUND_OUT
17	33	17: PPU_/RD	34	47: PPU_/WR
18	35	18: C1RAM_A10	36	48: C1RAM_/CE
19	37	19: PPU_A6	38	49: PPU_/A13
20	39	20: PPU_A5	40	50: PPU_A7
21	41	21: PPU_A4	42	51: PPU_A8
22	43	22: PPU_A3	44	52: PPU_A9

列	下の列(ヘッダー番号)	カセット番・信号	上の列(ヘッダー番号)	カセット番・信号
23	45	23: PPU_A2	46	53: PPU_A10
24	47	24: PPU_A1	48	54: PPU_A11
25	49	25: PPU_A0	50	55: PPU_A12
26	51	26: PPU_D0	52	56: PPU_A13
27	53	27: PPU_D1	54	57: PPU_D7
28	55	28: PPU_D2	56	58: PPU_D6
29	57	29: PPU_D3	58	59: PPU_D5
30	59	30: +5V	60	60: PPU_D4

J5 デバッグ(2x8)

ピン	信号	つながる先
1	PPU_/RD_3V3	P4-11(GPIO40), U2.18
2	PPU_/WR_3V3	P4-14(GPIO41), U2.17
3	PPU_A13_3V3	P4-13(GPIO42), U2.16
4	CIRAM_A10_MCU	JP1.1, P4-16(GPIO43)
5	EXP_AUDIO_PWM	P4-15(GPIO44), R9.1
6	EXP_AUDIO_RC	C3.1, R9.2, VR1.1
7	IRQ_DRV	P2-2(GPIO45), R4.1
8	FC_5V_SENSE	P2-4(GPIO46), R2.2, R3.1
9	+3V3	C2.1, P2-16(3V3), U2.1, U2.20
10	VBUS_MOD	C1.1, D1.1, J6.1, P2-15(VBUS)
11	CIRAM_A10	J1.18, JP1.2, JP2.2, JP3.2
12	GND	GND(全部品共通)
13	GND	GND(全部品共通)
14	GND	GND(全部品共通)
15	GND	GND(全部品共通)
16	GND	GND(全部品共通)

周辺部品の接続

部品-ピン	信号
U2-1 (DIR)	+3V3
U2-2 (A1)	PPU_/RD
U2-3 (A2)	PPU_/WR
U2-4 (A3)	PPU_A13
U2-5 (A4)	GND
U2-6 (A5)	GND
U2-7 (A6)	GND
U2-8 (A7)	GND
U2-9 (A8)	GND
U2-10 (GND)	GND
U2-11 (B8)	(未接続)

部品-ピン	信号
U2-12 (B7)	(未接続)
U2-13 (B6)	(未接続)
U2-14 (B5)	(未接続)
U2-15 (B4)	(未接続)
U2-16 (B3)	PPU_A13_3V3
U2-17 (B2)	PPU_/WR_3V3
U2-18 (B1)	PPU_/RD_3V3
U2-19 (/OE)	GND
U2-20 (VCC)	+3V3
Q1-1 (E)	GND
Q1-2 (C)	/IRQ
Q1-3 (B)	Q1_B
D1-1	VBUS_MOD
D1-2	+5V
C1-1	VBUS_MOD
C1-2	GND
C2-1	+3V3
C2-2	GND
C3-1	EXP_AUDIO_RC
C3-2	GND
C4-1	SOUND_OUT
C4-2	LOUT
R1-1	PPU_/A13
R1-2	CIRAM_/CE
R2-1	+5V
R2-2	FC_5V_SENSE
R3-1	FC_5V_SENSE
R3-2	GND
R4-1	IRQ_DRV
R4-2	Q1_B
R5-1	Q1_B
R5-2	GND
R6-1	+5V
R6-2	/IRQ
R7-1	SOUND_IN
R7-2	SOUND_OUT
R8-1	EXP_AUDIO_VR
R8-2	SOUND_OUT
R9-1	EXP_AUDIO_PWM
R9-2	EXP_AUDIO_RC
R10-1	LOUT
R10-2	GND

部品-ピン	信号
R11-1	LINE
R11-2	LOUT
JP1-1	CIRAM_A10_MCU
JP1-2	CIRAM_A10
JP2-1	PPU_A10
JP2-2	CIRAM_A10
JP3-1	PPU_A11
JP3-2	CIRAM_A10
SW1-1	BOOTSEL
SW1-2	GND
SW2-1	RUN
SW2-2	GND
J2-1 (SLEEVE)	GND
J2-2 (RING)	LINE
J2-3 (RING_SW)	(未接続)
J2-4 (TIP)	LINE
J2-5 (TIP_SW)	(未接続)
J3-1	SWCLK
J3-2	GND
J3-3	SWD
TP1-1 (TP)	PPU_/RD
TP2-1 (TP)	PPU_D0
TP3-1 (TP)	M2
TP4-1 (TP)	/ROMSEL
TP5-1 (TP)	GND

付録 B. ソフトウェアの規模

まとめり	ファイル	行
カセット(RP2350、C)	11	1,769
6502 プログラムを作る Python	6	824
Wi-Fi ブリッジ(ESP32-C6)	1	137
PC のアプリ・道具(Python / PowerShell)	11	1,785
PC の試験台(C、agnes を除く)	6	927
基板(KiCad を動かす Python)	22	2,456
書類を作るスクリプト	3	368

(生成したヘッダー(*_driver.h など)、agnes、ビルド済みのファイルは数えていない)

関係するファイル

ファイル	内容
README.md	リポジトリの説明とピン割り当て

ファイル	内容
docs/system_spec.pdf(この書類)、docs/build_system_spec.py、docs/system_spec_body.md	総合仕様書とその作り方
docs/hardware_spec.pdf、docs/hardware_spec_body.md	ハードウェア仕様書(3 章の元)
docs/user_manual.pdf	使い方の手順書
kicad/	回路図・基板・ガーバー(FC-MAGICON_gerber_r0.2.zip)・部品表・作るスクリプト
firmware/magicon/	カセットのファームウェア(main.c・cart.c・audio.c・PIO・ドライバーを作る Python)
firmware/bus_test/、chr_test/、usb_test/、host_test/	試験用のファームウェア
firmware/wifi_bridge/	ESP32-C6 の Wi-Fi ブリッジ
firmware/tools/	PC のアプリと道具
firmware/sim/	PC の試験台
firmware/out/	ビルド済みのファームウェア
3d/	STEP(部品あり / なし)
case/	ケースの調査とシール(case/label/)
images/	基板の 3D 画像とロゴ